Python 循环详解

从「看不懂」到「写得出」· 配 18 道由易到难的例题

这份手册怎么用

第一到第六节讲知识，每节后面跟着例题，**例题都有答案和讲解。**

第七节是分类题库，按难度分五级，从打印一行字到百钱百鸡。

第八、九节讲坑和写法，卡住的时候翻这两节。

建议顺序：先读第六节「状态管理」，那是最容易卡的地方，再回头顺着读。

一、循环到底是什么

循环就是把同一段代码重复执行。没有循环，程序只能从上往下走一遍；有了循环，程序才会「反复做事」。

1. 循环的三要素

要素	作用	对应代码
起点	从哪开始	`i = 1` 或 `range(1, 11)`
条件	什么时候继续	`i <= 10`
更新	每轮怎么变	`i += 1`

三样缺一，循环就不成立。for 循环把「起点、条件、更新」都藏在 `range()` 里了，所以看起来只有一行；while 循环三样都得你自己写。

2. for 和 while 怎么选

	for	while
适合	次数已知，或要遍历一个范围	次数未知，靠条件决定停不停
例子	打印 1 到 100、算 100 次方	猜数字直到猜中、输入 q 才退出
风险	基本没有	忘写更新就是死循环

一句话记：知道几次用 for，不知道几次用 while。拿不准的时候优先 for，因为它不容易写错。

3. range 的四种写法（必须记死）

写法	产生	说明
`range(5)`	0 1 2 3 4	从 0 开始，取不到 5
`range(1, 5)`	1 2 3 4	左闭右开，取不到 5
`range(1, 10, 3)`	1 4 7	步长 3
`range(100, 0, -50)`	100 50	负步长，倒着数，取不到 0

最常见的错：想数到 100，却写 `range(100)`，结果是 0 到 99。要么写 `range(1, 101)`，要么写 `range(100)` 心里清楚差一个。

4. 循环的执行顺序（拿小数字走一遍）

```
total = 0
for i in range(1, 4):
    total += i
print(total)
```

轮次	i 的值	执行后 total

第 1 轮	1	0 + 1 = 1
第 2 轮	2	1 + 2 = 3
第 3 轮	3	3 + 3 = 6
结束	—	打印 6

看不懂代码的时候，就这样画表走一遍。数字用 3、4 这种小的，脑子才跟得上。

二、for 循环

1. 基本形状

```
for 变量 in range(...):
    要做的事          # 缩进四格
```

- 行末必须有英文冒号。
- 下面缩进四格的行才属于循环，不缩进的在循环结束后执行一次。
- 循环变量（通常写 i）每轮自动换一个值，你不用管它怎么变。
- 用不到循环变量时，按惯例写成 `_`：`for _ in range(3):`。

2. 累加器模式（最高频的套路）

```
total = 0                 # ① 准备盒子，先装 0
for i in range(1, 101):
    total += i            # ② 每轮往里加
print(total)              # ③ 循环结束后打印
```

这三步行话叫「初始化、累加、输出」。忘记第 ③ 步，程序算完了但你看不见结果，这是新手最常见的漏。

例题 1　打印 1 到 10　Lv1

题目：用 for 循环打印 1 到 10，每行一个。

```
for i in range(1, 11):
    print(i)
```

要点：想打到 10，右边界要写 11。range 是左闭右开。

例题 2　1 加到 100　Lv1

题目：求 1+2+3+…+100 的和。

```
total = 0
for i in range(1, 101):
    total += i
print(total)        # 5050
```

要点：累加器一定先设 0。如果你想做乘法（阶乘），初值要设 1。

例题 3　打印偶数 Lv2

题目：打印 1 到 20 之间的偶数。

```
# 写法一：用条件筛
for i in range(1, 21):
    if i % 2 == 0:
        print(i)

# 写法二：直接用步长（更简洁）
for i in range(2, 21, 2):
    print(i)
```

要点：两种都对。能用步长解决的，优先步长，代码短一半。

例题 4　偶数求和 Lv2

题目：求 1 到 100 所有偶数的和。

```
total = 0
for i in range(2, 101, 2):
    total += i
print(total)        # 2550
```

要点：注意跟例题 2 的区别——加不加条件。

三、while 循环

1. 基本形状

```
i = 1              # ① 起点
while i <= 5:      # ② 条件
    print(i)
    i += 1         # ③ 更新，缺了就是死循环
```

死循环长什么样：程序一直跑不停，运行窗口刷屏。PyCharm 里点运行窗口左上角的红色方块能停，命令行里按 Ctrl+C。

2. while True + break 模式

有些场景你不知道要循环几次，只知道「什么时候可以结束」。这时写成 `while True`，在循环体里判断，该走就 break。

```
while True:
    cmd = input("输入点什么（输 q 退出）: ")
    if cmd == "q":
        break
    print("你输入的是", cmd)
```

要点：`while True` 不是死循环，只要有一条路能 break，它就是正常的。这类写法在猜数字、菜单程序里到处都是。

例题 5　while 累加　Lv2

题目：用 while 求 1 加到 100。

```
total = 0
i = 1
while i <= 100:
    total += i
    i += 1
print(total)        # 5050
```

要点：注意 `i += 1` 放在循环体里面，不是在 if 里面，也不要漏。

例题 6　倒数 Lv2

题目：用 while 从 10 数到 1，每行一个，最后打印「发射」。

```
i = 10
while i >= 1:
    print(i)
    i -= 1
print("发射")
```

要点：倒着数就把更新写成 `i -= 1`，条件写成 `i >= 1`。

四、break 和 continue

	break	continue
作用	立刻结束整个循环	跳过本轮剩下的代码，进入下一轮
比喻	掀桌子走人	这盘菜不吃，下一盘继续
之后	循环外第一行继续执行	循环继续转

例题 7　找到就停 Lv2

题目：找出 1 到 100 里第一个能被 7 整除的数。

```
for i in range(1, 101):
    if i % 7 == 0:
        print(i)      # 7
        break
```

要点：「找到就走」的场景，print 后面紧跟 break。不加 break 会把所有 7 的倍数都打出来。

例题 8　跳过某些轮次 Lv2

题目：打印 1 到 10，跳过 3 的倍数。

```
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i)          # 1 2 4 5 7 8 10
```

要点：写在 if 里、写 return 那种想法是不对的。continue 只是「这轮不做了」，循环不会停。

例题 9　break 和 continue 的区别（一定要自己跑）　Lv3

```
for i in range(5):
    if i == 2:
        break
    print("break 版：", i)     # 0 1

for i in range(5):
    if i == 2:
        continue
    print("continue 版：", i)  # 0 1 3 4
```

要点：break 把 2 后面的全砍了；continue 只砍掉 2 这一轮。

五、嵌套循环

循环里面还能再套循环。外层转一圈，内层转一整圈。摸清顺序就一句话：**外层管行，内层管列。**

```
for i in range(1, 4):        # 外层：3 行
    for j in range(1, 3):    # 内层：每行 2 个
        print(i, j)
    print("---")
```

输出：

```
1 1
1 2
---
2 1
2 2
---
3 1
3 2
---
```

看这个输出，能明白「外层慢、内层快」。i 还是 1 的时候，j 已经跑完 1、2 两轮了。

例题 10　打印星号三角形 Lv3

题目：打印 5 行，第 1 行 1 个星，第 2 行 2 个，一直到 5 个。

```
# 写法一：字符串乘法，一行搞定
for i in range(1, 6):
    print("*" * i)

# 写法二：内层循环，更通用
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()
```

要点：`end=""` 让 print 不换行，一行打完了来一个空的 `print()` 换行。

例题 11　九九乘法表 Lv4

```
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f"{i}×{j}={i*j}", end="\t")
    print()
```

要点：内层到 `i + 1` 为止，所以第 3 行只打到 3×3。这就是三角形效果。

例题 12　金字塔 Lv4

题目：打印 5 行等腰三角形。

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

要点：每行 = 空格数 + 星号数。空格 `n - i`，星号 `2*i - 1`。先想清这两个公式再写。

六、状态管理（最容易卡的地方）

很多人「看得懂但写不出」，卡的不是循环语法，是**循环里那些会变的变量**。循环里一共只有四种状态，认全了就通了。

类型	干什么	初值	循环里怎么写
计数器	数一共发生了几次	`count = 0`	`count += 1`
累加器	把值一点点加起来	`total = 0`	`total += 值`
最值	记住目前最大/最小的那个	第一个元素	`if 新值 > 最大: 最大 = 新值`
标志位	记一件事有没有发生过	`flag = True/False`	发生就改，必要时 break

1. 计数器

例题 13　数偶数个数 Lv2

```
count = 0
for i in range(1, 101):
    if i % 2 == 0:
        count += 1
print(count)        # 50
```

要点：计数器只在条件成立时加 1，不是每轮都加。

2. 最值

例题 14　找最大数 Lv3

```
big = int(input("第1个数："))
for i in range(4):
    n = int(input("下一个数："))
    if n > big:
        big = n
print("最大的是", big)
```

要点：先把第一个数当作「目前最大」，后面挨个比。这个套路能解决「最大值、最小值、最高分」一大类题。

3. 标志位（素数就是它）

例题 15　判断素数 Lv3

```
num = int(input("请输入一个大于1的整数："))
is_prime = True                 # 先假设它是素数
for i in range(2, int(num ** 0.5) + 1):
    if num % i == 0:
        is_prime = False        # 找到因子，推翻假设
        break
if is_prime:                    # 判断放在循环外面！
    print(f"{num} 是素数")
else:
    print(f"{num} 不是素数")
```

千万别这样写：把 if/else 放进循环里。
那样每试一个数就打印一次，结果反反复复，输出一堆「是/不是」。
正确做法：循环里只负责改标志，判断和打印都放到循环外面。

4. 边循环边记录

例题 16　找质数并统计 Lv4

题目：找出 1 到 100 所有质数，打印出来，并统计个数。

```
count = 0
for num in range(2, 101):
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, end=" ")
        count += 1
print()
print("一共", count, "个")
```

要点：标志位要放在外层循环里，每换一个数就重置一次。这是嵌套 + 标志的组合。

七、分类题库（18 题，答案都在下面）

建议做法：先不看答案，自己写。写完再对。写不出来就退回中文步骤。

Lv1　热手

1. 打印 1 到 10，每行一个。
2. 打印 10 到 1，每行一个。
3. 打印 1 到 10 的平方，格式「3 的平方是 9」。

Lv2　加条件

4. 求 1 到 100 的和。
5. 求 1 到 100 里所有奇数的和。
6. 打印 1 到 100 里能被 7 整除的数。
7. 数一数 1 到 100 里有多少个数能被 3 整除。

Lv3　状态管起来

8. 输入 5 个数，找出最小的。
9. 输入一个数，判断它是不是素数。
10. 求 1 到 100 里能同时被 3 和 5 整除的数的和。
11. 输入一个三位数，输出各位数字之和（385 → 16）。

Lv4　套起来

12. 打印九九乘法表。
13. 打印 5 行倒三角（5 4 3 2 1 个星）。
14. 打印斐波那契数列前 10 项（1 1 2 3 5 8...）。

Lv5　综合

15. 猜数字：电脑出 1 到 100，最多猜 5 次，用完提示答案。
16. 找出 100 到 999 之间所有水仙花数（各位立方和等于本身，如 153）。
17. 百钱百鸡：公鸡 5 元、母鸡 3 元、小鸡 1 元 3 只，100 元买 100 只，求组合。
18. 求 1 到 100 里所有质数的和。

题库答案

1. 打印 1 到 10

```
for i in range(1, 11):
    print(i)
```

2. 打印 10 到 1

```
for i in range(10, 0, -1):
    print(i)
```

3. 平方

```
for i in range(1, 11):
    print(f"{i} 的平方是 {i ** 2}")
```

4. 1 到 100 的和

```
total = 0
for i in range(1, 101):
    total += i
print(total)        # 5050
```

5. 奇数和

```
total = 0
for i in range(1, 101, 2):
    total += i
print(total)        # 2500
```

6. 7 的倍数

```
for i in range(1, 101):
    if i % 7 == 0:
        print(i)
```

7. 数 3 的倍数个数

```
count = 0
for i in range(1, 101):
    if i % 3 == 0:
        count += 1
print(count)        # 33
```

8. 找最小

```
small = int(input("第1个数： "))
for i in range(4):
    n = int(input("下一个： "))
    if n < small:
```

```
        small = n
print("最小的是", small)
```

9. 判断素数

```
num = int(input("请输入一个大于1的整数："))
is_prime = True
for i in range(2, int(num ** 0.5) + 1):
    if num % i == 0:
        is_prime = False
        break
print(f"{num} 是素数" if is_prime else f"{num} 不是素数")
```

10. 3 和 5 的公倍数之和

```
total = 0
for i in range(1, 101):
    if i % 3 == 0 and i % 5 == 0:
        total += i
print(total)        # 315
```

注意：算完一定要 print(total)，只算不打印，屏幕上什么都没有。

11. 三位数各位之和

```
n = int(input("请输入三位数："))
bai = n // 100
shi = n // 10 % 10
ge = n % 10
print(bai + shi + ge)
```

凭感觉写也行，但这个写法最干净：百位用 //100，十位用 //10 再 %10，个位直接 %10。

12. 九九乘法表

```
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f"{i}×{j}={i*j}", end="\t")
    print()
```

13. 倒三角

```
for i in range(5, 0, -1):
    print("*" * i)
```

14. 斐波那契前 10 项

```
a, b = 1, 1
for i in range(10):
    print(a, end=" ")
    a, b = b, a + b
```

`a, b = b, a + b` 是同时赋值，右边先全算完再装左边。写成两句就错了。

15. 猜数字最多 5 次

```
import random

answer = random.randrange(1, 101)
for i in range(5):
    guess = int(input(f"第{i+1}次猜：  "))
    if guess == answer:
        print("猜对了")
        break
    elif guess > answer:
        print("大了")
    else:
        print("小了")
else:
    print(f"机会用完了，答案是 {answer}")
```

for-else：循环没被 break 打断才会执行 else。不想用这个特性，就自己加个计数器。

16. 水仙花数

```
for n in range(100, 1000):
    a = n // 100
    b = n // 10 % 10
    c = n % 10
    if a ** 3 + b ** 3 + c ** 3 == n:
        print(n)
```

17. 百钱百鸡

```
for cock in range(0, 21):
    for hen in range(0, 34):
        chick = 100 - cock - hen
        if chick % 3 == 0 and 5 * cock + 3 * hen + chick // 3 == 100:
            print(f"公鸡{cock}只，母鸡{hen}只，小鸡{chick}只")
```

18. 质数之和

```
total = 0
for num in range(2, 101):
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        total += num
print(total)        # 1060
```

八、循环里最常见的九个坑

现象	原因	怎么修
程序一直跑不停	while 里忘了让变量变化	循环体里加 `i += 1`
少跑一次 / 多跑一次	range 右边界是开区间	想含 100 就写 `range(1, 101)`
屏幕上什么都没有	算完了没有 print	循环结束后补 `print(total)`
打出一堆重复结果	该在循环外判断的放进了循环	把 if 判断搬到循环外面
找到答案了还在跑	忘了 break	print 后面加 `break`
IndentationError	缩进不一致	同一层统一四空格
SyntaxError	中文冒号、漏冒号、单等号	切英文输入法，检查 : 和 ==
TypeError: ... str and int	文字跟数字拼接或比较	先 int() 转，再算
只打印了最后一个	print 写在了循环外面	要每轮都打印，就缩进进循环体

九、写不出来的解法：五步

1. **先写中文步骤。**三到五句，每句一个动作，不许出现代码。
2. **定状态。**问自己四件事：数据从哪来？要几个变量、初值多少？循环里谁在变？最后打印什么？
3. **把中文翻成代码。**一句对一行，不许合并。
4. **先跑起来再加功能。**先让它打印一行，跑通，再加条件、再加累加。
5. **报错了就读最后一行。**它一般直接告诉你哪里错了（哪行、什么类型）。

例子：求 1 到 100 里能被 7 整除的数的和

① 中文步骤：
准备一个 total 装 0 → 让 i 从 1 走到 100 → 每次看 i 能不能被 7 整除 → 能加就放进 total → 走完打印 total

② 四问：数据来自 1 到 100 的范围；变量两个（total、i）；循环里变的是 total；输出 total。

③ 翻译：

```
total = 0
for i in range(1, 101):
    if i % 7 == 0:
        total += i
print(total)
```

最后一句

循环写不出来，九成不是语法问题，是「状态」没想清楚。

把计数器、累加器、最值、标志位这四样摆出来，对着题问「这题需要哪几个」，答案就出来了。